



SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

OPERATING SYSTEMS LABORATORY

Course Code: CSE309R01

Semester: V

Lab Manual

2026

ROHITH KANNA GV

128003214

A-44

**SCHOOL OF COMPUTING
SHANMUGHA ARTS, SCIENCE, TECHNOLOGY AND RESEARCH
ACADEMY
(SASTRA Deemed to be University)
Tirumalaisamudram, Thanjavur-613 401**

Exercise No. 11: Implementation of Banker's Algorithm for Deadlock

Avoidance Write a program to simulate banker's algorithm for deadlock

avoidance **Objectives**

The primary goal of the Banker's Algorithm is to prevent deadlock while allocation of resources to processes, where multiple processes compete for limited resources (such as CPU time, memory, or devices).

Theory or Concept

The algorithm ensures that the system remains in a safe state during resource allocation.

A safe state allows all processes to complete their execution without getting stuck due to resource unavailability.

Procedure / Algorithm

Step 1: Input the values of Max, Allocation and Resources arrays

Step 2: Calculate values of Need and Available

Step 3: Implement safety algorithm to determine whether the system is in safe state or not **Step 4:** Input additional resource request of process

Step 5: If request > Max then report error

Step 6: If request > Available then return resource unavailable

Step 7: Allocate resource as per request and invoke safety algorithm to determine whether the system is in safe state or not

Step 8: If safe state then requests accepted else request rejected

Sample Input / Output

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2
P ₁	2 0 0	3 2 2	
P ₂	3 0 2	9 0 2	
P ₃	2 1 1	2 2 2	
P ₄	0 0 2	4 3 3	

Request 1: P2 makes a request for 1 additional unit of C

Request 2: P0 requests one unit of A and C

CODE:

```
#include <stdio.h>

#define P 5 // Number of processes
#define R 3 // Number of resource types

// Global matrices and arrays
int max[P][R], alloc[P][R], need[P][R], available[R];

// Function to calculate the Need matrix
void calculateNeed() {
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
}

// Safety Algorithm to check if system is in a safe state
int isSafe() {
    int finish[P] = {0};
    int work[R];

    // Initialize work = available
    for (int i = 0; i < R; i++) {
        work[i] = available[i];
    }

    int safeSequence[P];
    int count = 0;

    while (count < P) {
        int found = 0;
        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                int canAllocate = 1;
                for (int j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {
                        canAllocate = 0;
                        break;
                    }
                }
                if (canAllocate) {
                    // Assume process i completes, release resources
                    for (int j = 0; j < R; j++) {
                        work[j] += alloc[i][j];
                    }
                    finish[i] = 1;
                    safeSequence[count] = i;
                }
            }
        }
    }
}
```

```

        count++;
        found = 1;
    }
}
}
if (!found) {
    // No process could be allocated resources in this pass
    return 0; // Not safe
}

// If loop completes, system is in a safe state
printf("System is in a safe state.\nSafe sequence: ");
for (int i = 0; i < P; i++) {
    printf("P%d ", safeSequence[i]);
}
printf("\n");
return 1;
}

// Function to handle resource request from a process
int requestResources(int process, int request[R]) {
    // Check if request exceeds need
    for (int j = 0; j < R; j++) {
        if (request[j] > need[process][j]) {
            printf("Error: Process P%d has exceeded its maximum
claim.\n", process);
            return 0;
        }
    }

    // Check if resources are available
    for (int j = 0; j < R; j++) {
        if (request[j] > available[j]) {
            printf("Resources unavailable for P%d. Process must wait.\n",
process);
            return 0;
        }
    }

    // Try to allocate resources temporarily
    for (int j = 0; j < R; j++) {
        available[j] -= request[j];
        alloc[process][j] += request[j];
        need[process][j] -= request[j];
    }

    // Check system safety after allocation
    if (isSafe()) {

```

```

        printf("Resources allocated to P%d.\n", process);
        return 1;
    } else {
        // Rollback
        for (int j = 0; j < R; j++) {
            available[j] += request[j];
            alloc[process][j] -= request[j];
            need[process][j] += request[j];
        }
        printf("Process P%d must wait, system not in safe state.\n",
process);
        return 0;
    }
}

int main() {
    // Example initialization
    // Max matrix
    int maxInput[P][R] = {
        {7, 5, 3},
        {3, 2, 2},
        {9, 0, 2},
        {2, 2, 2},
        {4, 3, 3}
    };

    // Allocation matrix
    int allocInput[P][R] = {
        {0, 1, 0},
        {2, 0, 0},
        {3, 0, 2},
        {2, 1, 1},
        {0, 0, 2}
    };

    // Available resources
    int availInput[R] = {3, 3, 2};

    // Assign to global variables
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            max[i][j] = maxInput[i][j];
            alloc[i][j] = allocInput[i][j];
        }
    }
    for (int j = 0; j < R; j++) {
        available[j] = availInput[j];
    }
}

```

```

// Calculate Need matrix
calculateNeed();

// Check initial system safety
isSafe();

// Example resource requests
int request1[R] = {1, 0, 2}; // P1 requests
int request2[R] = {0, 2, 0}; // P0 requests

printf("\nProcess P1 requesting {1, 0, 2}\n");
requestResources(1, request1);

printf("\nProcess P0 requesting {0, 2, 0}\n");
requestResources(0, request2);

return 0;
}

```

Output:

```

guest-gylmek@cnlab:~$ gedit bank.c
guest-gylmek@cnlab:~$ gcc bank.c
guest-gylmek@cnlab:~$ ./a.out
System is in a safe state.
Safe sequence: P1 P3 P4 P0 P2

Process P1 requesting {1, 0, 2}
System is in a safe state.
Safe sequence: P1 P3 P4 P0 P2
Resources allocated to P1.

Process P0 requesting {0, 2, 0}
Process P0 must wait, system not in safe state.

```

Observation

Solve a problem using bankers' algorithm and check for the possibility of deadlock. If there is no deadlock generate the safe sequence.

The system's initial state was verified using the safety algorithm, confirming it is in a safe state with no possibility of deadlock. When evaluating subsequent resource

allocations, Request 1 (P2) is denied as it exceeds the process's maximum declared claim, and Request 2 (P0) is deferred because it leaves the system in an unsafe state.

Result:

The system is safe, and the generated initial safe sequence is: **P1 → P3 → P4 → P0 → P2.**

Would you like me to format this directly into a **standard lab report structure** (including Aim, Algorithm, and Conclusion) that you can copy into your record book?